MIT 8103 Advanced Database Systems: Portfolio Summary

Alliance Chemicals Global IT Procurement & Infrastructure Management System

Name: Anifowoshe Emmanuel

Student ID: 301929334

GitHub: https://github.com/biyidevops/Anifowoshe_Emmanuel_301929334

Case Study

The IT procurement system records staff, hardware assets, vendors, and assignments. IT staff assign hardware to employees, track returns, and monitor inventory. This example covers Alliance Chemicals Global using fictional records. PostgreSQL holds the relational inventory and assignments; MongoDB holds sample deployment and return events. Both run in one GitHub Codespace.

Portfolio 1: Database Design and Modelling

The relational design includes staff, asset, vendor, procurementorder, and assetassignment tables. Separating assets from staff and vendors allows independent management. Each table has a primary key, with foreign keys connecting assignments to staff/assets, and assets to procurement orders. Constraints reject invalid dates. A unique partial index enforces one open assignment per asset. Normalisation converts a combined list to 1NF (one assignment per row), separates asset details in 2NF, and isolates vendor/staff details in 3NF. The schema, ER diagram, and rejected invalid inserts are saved in Portfolio 1.

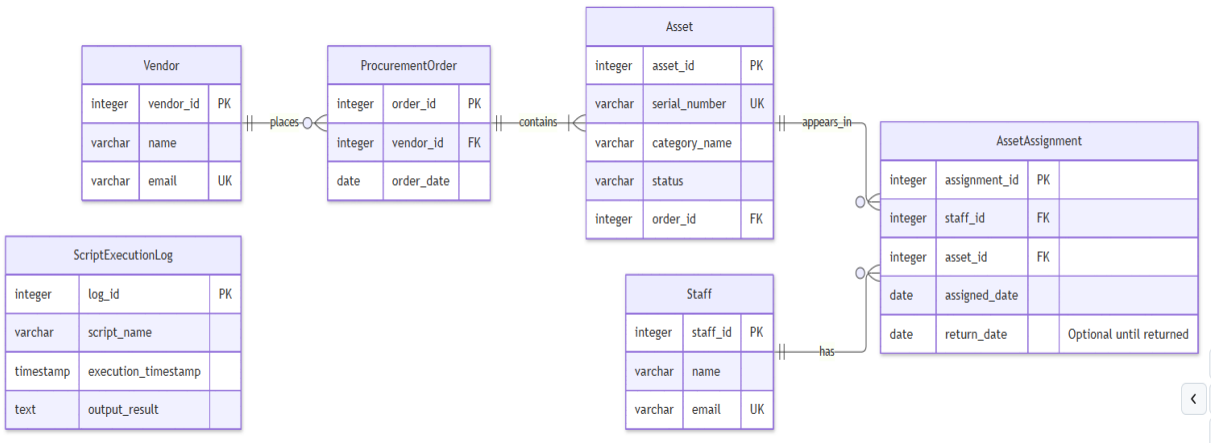


Figure 1. *The six database tables and their relationships, where PK identifies a unique row and FK links tables together. Each asset assignment connects one staff member to one specific asset, while each asset links back to a procurement order and vendor. The standalone script execution log tracks automated administrative tasks, while a staff member, vendor, or order can have zero or many related records within the core system.*

Portfolio 2: Query Processing and Optimisation

Three queries retrieve open assignments, available inventory, and assignment counts per staff member. James Babalola and Emmanuel Anifowoshe have open assignments; one router remains unassigned. A separate table, p2_benchmark.assetassignment_history, contains 100,000 generated closed-assignment records for speed testing. A date-range count returns 1,674 rows before and after adding an index on assigned_date. EXPLAIN (ANALYZE, BUFFERS) reveals the execution plan. After warm-ups, the third run's Sequential Scan took 5.850 ms with 736 shared buffer hits. With the index, it used a Bitmap Heap Scan taking only 0.415 ms and 74 shared buffer hits. The index efficiently avoids scanning unrelated dates, though these cached Codespace measurements do not predict all workloads.

Portfolio 3: Transactions and Concurrency

A hardware return is saved with COMMIT. In another transaction, a valid assignment is followed by an invalid date; ROLLBACK removes the unsaved assignment while keeping the committed return. Two sessions then use READ COMMITTED isolation. Session A inserts an assignment without committing. Session B tries to assign the same asset but waits. After Session A commits, Session B receives a duplicate-key error from the unique partial index and rolls back. This demonstrates why availability checks alone are insufficient and how the unique index enforces the one-open-assignment rule during concurrent inserts.

Portfolio 4: NoSQL and Advanced Data Models

MongoDB stores four documents in it_events.asset_events. Each embeds staff and asset details, a date, and event-specific details (deployment location or hardware condition), keeping event information self-contained. The insert is acknowledged for four events (EVT001-EVT004). Querying SN001 returns its initial assignment, return, and reassignment. Searching details.condition for "requires maintenance" returns EVT002. These are separately entered sample events, not a synchronized audit log. PostgreSQL remains responsible for relational constraints.

Portfolio 5: Distributed and Cloud Database Exercise

The proposal suggests three voting, data-holding MongoDB replica members across separate failure zones (Lagos HQ, Nairobi Branch, Cloud Region). A primary accepts writes; secondaries copy changes. If the primary fails, a reachable majority elects a replacement, pausing writes briefly. Majority acknowledgement and secondary lag affect durability and recent-event reads. Sharding divides records, which our small sample volume does not justify. A managed cloud deployment requires authentication, encryption, restricted access, and backups. Cloud replication does not protect or synchronize PostgreSQL automatically. This remains a proposed design; no replica set or failover was implemented.

Evidence

Scripts, explanations, and screenshots are provided for P1-P5. Earlier screenshots record database states before Portfolio 3 changed the assignments.

Individual reflection

For MongoDB, I embedded staff names and hardware serial numbers directly inside the asset_events document rather than referencing other collections. This ensures the event log acts as an immutable snapshot, preserving the exact assignee details even if the relational database changes. During Portfolio 3, managing simultaneous database connections was challenging. I accidentally ran commands in the wrong terminal, causing lock wait states to fail. I fixed this by slowing down, clearly separating my terminal windows, and executing SQL line-by-line. I learned the critical importance of patience and execution order when testing transactional locks.

References

[1] PostgreSQL Global Development Group. PostgreSQL 16: Using EXPLAIN. https://www.postgresql.org/docs/16/using-explain.html

[2] PostgreSQL Global Development Group. PostgreSQL 16: Transaction Isolation. https://www.postgresql.org/docs/16/transaction-iso.html

[3] MongoDB. Data Modeling in MongoDB. https://www.mongodb.com/docs/manual/data-modeling/

[4] MongoDB. Replication. https://www.mongodb.com/docs/manual/replication/

[5] MongoDB. Sharding. https://www.mongodb.com/docs/manual/sharding/

AI-use declaration

I used Google Gemini and GitHub Copilot to explain database concepts, troubleshoot Codespaces errors, and review the structure and wording of my portfolio. I executed the database commands myself, verified the query outputs, edited the SQL schemas to fit my case study, and checked the results against the assessment requirements before submission. I understand the work and remain responsible for its accuracy.